



INSTITUTO POLITÉCNICO NACIONAL



INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

SISTEMAS EN CHIP

Reporte de la práctica 6

“Contador sin rebotes”

Grupo: 6CM1

Integrantes

- García Escamilla Bryan Alexis
- Meléndez Macedonio Rodrigo

Profesor

Aguilar Sánchez Fernando

Fecha de entrega: 13 de febrero de 2025

INTRODUCCIÓN

Contadores sin rebotes

Cuando hablamos de contadores sin rebotes (Debounced Counters), entramos en un área donde el mundo físico choca con la velocidad extrema de la electrónica digital.

Si conectas un botón directamente a la entrada de un contador, veremos que, al presionarlo una vez, el contador aumenta 5, 10 o incluso 100 veces. Esto sucede por el fenómeno del rebote.

El rebote mecánico

Un interruptor o botón está hecho de piezas metálicas. Cuando presionas el botón, estas piezas no hacen un contacto perfecto de inmediato; en su lugar, chocan y "rebotan" mecánicamente durante unos pocos milisegundos antes de quedarse quietas.

Para un microcontrolador como el ATmega8535 o un contador digital, que pueden procesar millones de señales por segundo, cada uno de esos rebotes parece un pulso de reloj válido. El resultado es un conteo errático.

Soluciones por hardware (analógicas)

Existen dos formas clásicas de limpiar la señal antes de que llegue al contador:

- **Filtro RC (Resistor-Capacitor):** Se coloca una resistencia y un condensador que actúan como un "amortiguador". El condensador absorbe los picos rápidos de voltaje de los rebotes y entrega una señal suave.
- **Trigger de Schmitt (Schmitt Trigger):** Se usa para "cuadrar" la señal que sale del filtro RC. Este componente tiene histéresis, lo que significa que ignora pequeñas variaciones de voltaje y solo cambia de estado cuando se superan umbrales específicos. El chip 74LS14 es el ejemplo más común.
- **Latch SR (Bistable):** Es la solución definitiva por hardware. Usando un interruptor de dos posiciones y un par de compuertas NAND, el primer contacto "enclava" la salida y los rebotes posteriores son ignorados por completo.

Soluciones por software (digitales)

En proyectos con CodeVisionAVR, lo más común es ahorrar componentes y hacer el "anti-rebote" (debouncing) mediante código.

- **Retardo (Delay):** Es la técnica más simple. Cuando el microcontrolador detecta un cambio en el pin, espera unos 20 o 50 milisegundos (tiempo suficiente para que terminen los rebotes) y vuelve a leer el pin. Si sigue presionado, lo cuenta como un pulso válido.

- **Contador de Verificación:** En lugar de un retardo ciego, el software lee el pin en intervalos cortos (por ejemplo, cada 1 milisegundo). Solo si el pin mantiene el mismo valor durante 10 lecturas consecutivas, se considera una pulsación real.

Importancia en sistemas industriales

En el área de automatización, no tener un contador sin rebotes puede causar accidentes o errores graves de inventario (por ejemplo, contar 10 botellas cuando solo pasó una por la banda transportadora). Por eso, los PLC y microcontroladores de grado industrial incluyen filtros de entrada configurables.

DESARROLLO

Circuito en hecho en Proteus

Para el desarrollo de la práctica 6, el circuito a armar es el siguiente:

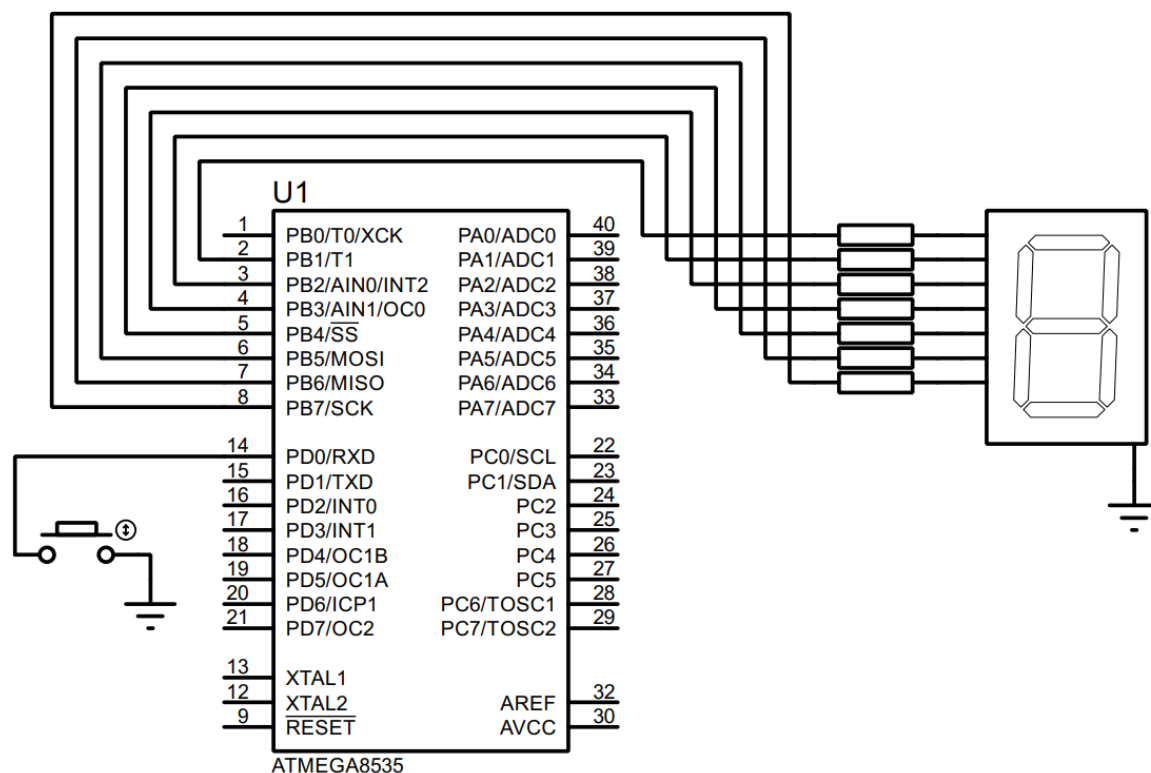


Imagen 1. Circuito creado en Proteus.

Configuración previa al código

De acuerdo con la imagen del circuito, el registro B se configura como de salida (PB0 – PB6) mientras que el registro D como de entrada (PD0), activando la resistencia de pull-up.

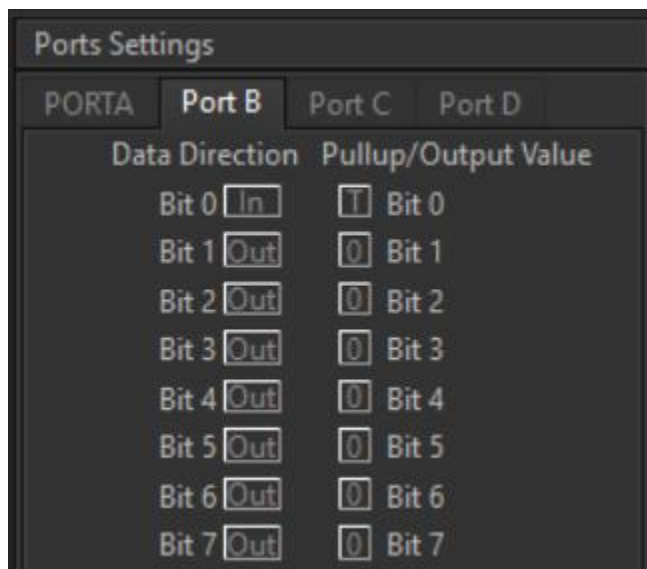


Imagen 2. Configuración del puerto B.

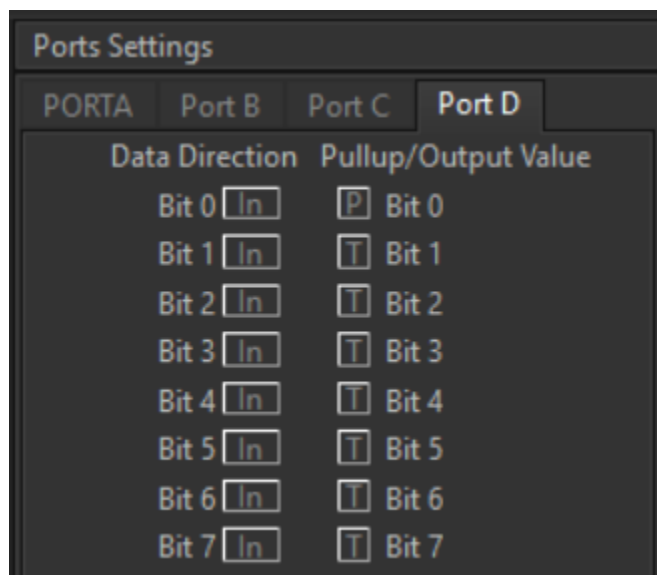


Imagen 3. Configuración del puerto D.

Para poder hacer el contador es necesario interpretar el número correspondiente del display como un número hexadecimal, para ello se realizó la siguiente tabla, tomando en cuenta lo que se indicó en la práctica 3:

Número Display	Combinaciones								Valor hexadecimal
	g	f	e	d	c	b	a		
0	0	1	1	1	1	1	1	0	0x7e
1	0	0	0	0	1	1	0	0	0x0c
2	1	0	1	1	0	1	1	0	0xb6
3	1	0	0	1	1	1	1	0	0x9e
4	1	1	0	0	1	1	0	0	0xcc

5	1	1	0	1	1	0	1	0	0xda
6	1	1	1	1	1	0	1	0	0xfa
7	0	0	0	0	1	1	1	0	0x0e
8	1	1	1	1	1	1	1	0	0xfe
9	1	1	0	1	1	1	1	0	0xde
A	1	1	1	0	1	1	1	0	0xee
B	1	1	1	1	1	0	0	0	0xf8
C	0	1	1	1	0	0	1	0	0x72
D	1	0	1	1	1	1	0	0	0xbc
E	1	1	1	1	0	0	1	0	0xf2
F	1	1	1	0	0	0	1	0	0xe2

Tabla 1. Representaci3n hexadecimal para los n3meros del 0 al 9 y del A a la F en hexadecimal.

C3digo del circuito de CodeVision

```

1  /*****
2  This program was created by the CodeWizardAVR V4.06a
3  Automatic Program Generator
4  Copyright 1998-2025 Pavel Haiduc, HP InfoTech S.R.L.
5  http://www.hpinfotech.ro
6
7  Project : PrÃ¡ctica 6 'Contador sin rebotes'
8  Version : 1.0
9  Date    : 12/02/2026
10 Author  : GarcÃ-a Escamilla Bryan Alexis
11 Company :
12 Comments:
13
14
15 Chip type      : ATmega8535
16 Program type   : Application
17 AVR Core Clock frequency: 1.000000 MHz
18 Memory model   : Small
19 External RAM size : 0
20 Data Stack size : 128
21 *****/
22
23 // I/O Registers definitions
24 #include <mega8535.h>
25 #include <delay.h>
26 #define BOTON PIND.0

```

```

28 // Declare your global variables here
29
30 void main(void) {
31     // Declare your local variables here
32     bit estado_boton_anterior, estado_boton_actual;
33     unsigned char numero_bcd;
34     const char bcd[10] = {0x7e, 0x0c, 0xb6, 0x9e, 0xcc, 0xda, 0xfa, 0x0e, 0xfe, 0xde};
35
36     // Input/Output Ports initialization
37     // Port A initialization
38     // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
39     DDRA=(0<<DDA7) | (0<<DDA6) | (0<<DDA5) | (0<<DDA4) | (0<<DDA3) | (0<<DDA2) | (0<<DDA1) | (0<<DDA0);
40     // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
41     PORTA=(0<<PORTA7) | (0<<PORTA6) | (0<<PORTA5) | (0<<PORTA4) | (0<<PORTA3) | (0<<PORTA2) | (0<<PORTA1) | (0<<PORTA0);
42
43     // Port B initialization
44     // Function: Bit7=Out Bit6=Out Bit5=Out Bit4=Out Bit3=Out Bit2=Out Bit1=Out Bit0=Out
45     DDRB=(1<<ddb7) | (1<<ddb6) | (1<<ddb5) | (1<<ddb4) | (1<<ddb3) | (1<<ddb2) | (1<<ddb1) | (1<<ddb0);
46     // State: Bit7=0 Bit6=0 Bit5=0 Bit4=0 Bit3=0 Bit2=0 Bit1=0 Bit0=0
47     PORTB=(0<<PORTB7) | (0<<PORTB6) | (0<<PORTB5) | (0<<PORTB4) | (0<<PORTB3) | (0<<PORTB2) | (0<<PORTB1) | (0<<PORTB0);
48
49     // Port C initialization
50     // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
51     DDRC=(0<<DDC7) | (0<<DDC6) | (0<<DDC5) | (0<<DDC4) | (0<<DDC3) | (0<<DDC2) | (0<<DDC1) | (0<<DDC0);
52     // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
53     PORTC=(0<<PORTC7) | (0<<PORTC6) | (0<<PORTC5) | (0<<PORTC4) | (0<<PORTC3) | (0<<PORTC2) | (0<<PORTC1) | (0<<PORTC0);
54
55     // Port D initialization
56     // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
57     DDRD=(0<<DDD7) | (0<<DDD6) | (0<<DDD5) | (0<<DDD4) | (0<<DDD3) | (0<<DDD2) | (0<<DDD1) | (0<<DDD0);
58     // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=P
59     PORTD=(0<<PORTD7) | (0<<PORTD6) | (0<<PORTD5) | (0<<PORTD4) | (0<<PORTD3) | (0<<PORTD2) | (0<<PORTD1) | (1<<PORTD0);
60
61     // Timer/Counter 0 initialization
62     // Clock source: System Clock
63     // Clock value: Timer 0 Stopped
64     // Mode: Normal top=0xFF
65     // OC0 output: Disconnected
66     TCCR0=(0<<WGM00) | (0<<COM01) | (0<<COM00) | (0<<WGM01) | (0<<CS02) | (0<<CS01) | (0<<CS00);
67     TCNT0=0x00;
68     OCR0=0x00;
69
70     // Timer/Counter 1 initialization
71     // Clock source: System Clock
72     // Clock value: Timer1 Stopped
73     // Mode: Normal top=0xFFFF
74     // OC1A output: Disconnected
75     // OC1B output: Disconnected
76     // Noise Canceler: Off
77     // Input Capture on Falling Edge
78     // Timer1 Overflow Interrupt: Off
79     // Input Capture Interrupt: Off
80     // Compare A Match Interrupt: Off
81     // Compare B Match Interrupt: Off
82     TCCR1A=(0<<COM1A1) | (0<<COM1A0) | (0<<COM1B1) | (0<<COM1B0) | (0<<WGM11) | (0<<WGM10);
83     TCCR1B=(0<<ICNC1) | (0<<ICES1) | (0<<WGM13) | (0<<WGM12) | (0<<CS12) | (0<<CS11) | (0<<CS10);
84     TCNT1H=0x00;
85     TCNT1L=0x00;
86     ICR1H=0x00;
87     ICR1L=0x00;
88     OCR1AH=0x00;
89     OCR1AL=0x00;
90     OCR1BH=0x00;
91     OCR1BL=0x00;

```



```

93 // Timer/Counter 2 initialization
94 // Clock source: System Clock
95 // Clock value: Timer2 Stopped
96 // Mode: Normal top=0xFF
97 // OC2 output: Disconnected
98 ASSR=(0<<AS2);
99 TCCR2=(0<<WGM20) | (0<<COM21) | (0<<COM20) | (0<<WGM21) | (0<<CS22) | (0<<CS21) | (0<<CS20);
100 TCNT2=0x00;
101 OCR2=0x00;
102
103 // Timer(s)/Counter(s) Interrupt(s) initialization
104 TIMSK=(0<<OCIE2) | (0<<TOIE2) | (0<<TICIE1) | (0<<OCIE1A) | (0<<OCIE1B) | (0<<TOIE1) | (0<<OCIE0) | (0<<TOIE0);
105
106 // External Interrupt(s) initialization
107 // INT0: Off
108 // INT1: Off
109 // INT2: Off
110 MCUCR=(0<<ISC11) | (0<<ISC10) | (0<<ISC01) | (0<<ISC00);
111 MCUCSR=(0<<ISC2);
112
113 // USART initialization
114 // USART disabled
115 UCSRB=(0<<RXCIE) | (0<<TXCIE) | (0<<UDRIE) | (0<<RXEN) | (0<<TXEN) | (0<<UCSZ2) | (0<<RXB8) | (0<<TXB8);
116
117 // Analog Comparator initialization
118 // Analog Comparator: Off
119 // The Analog Comparator's positive input is
120 // connected to the AIN0 pin
121 // The Analog Comparator's negative input is
122 // connected to the AIN1 pin
123 ACSR=(1<<ACD) | (0<<ACBG) | (0<<ACO) | (0<<ACI) | (0<<ACIE) | (0<<ACIC) | (0<<ACIS1) | (0<<ACIS0);
124 SFIOR=(0<<ACME);
125
126 // ADC initialization
127 // ADC disabled
128 ADCSRA=(0<<ADEN) | (0<<ADSC) | (0<<ADATE) | (0<<ADIF) | (0<<ADIE) | (0<<ADPS2) | (0<<ADPS1) | (0<<ADPS0);
129
130 // SPI initialization
131 // SPI disabled
132 SPCR=(0<<SPIE) | (0<<SPE) | (0<<DORD) | (0<<MSTR) | (0<<CPOL) | (0<<CPHA) | (0<<SPR1) | (0<<SPR0);
133
134 // TWI initialization
135 // TWI disabled
136 TWCR=(0<<TWEA) | (0<<TWSTA) | (0<<TWSTO) | (0<<TWEN) | (0<<TWIE);

```

```
138 while (1) {
139     // Place your code here
140     // Determina el estado de 'estado_boton_actual'
141     if (!BOTON) {
142         estado_boton_actual = 0;
143     } else {
144         estado_boton_actual = 1;
145     }
146
147     // Detectar el cambio de flanco de 1 a 0
148     if (estado_boton_anterior && !estado_boton_actual) {
149         numero_bcd++;
150
151         // Se reinicia cuando el conteo llega a 10
152         if (numero_bcd == 10) {
153             numero_bcd = 0;
154             delay_ms(40); // Eliminar rebotes
155         }
156     }
157
158     // Detectar el cambio de flanco de 0 a 1
159     if (!estado_boton_anterior && estado_boton_actual) {
160         delay_ms(40); // Eliminar rebotes
161     }
162
163     // Actualizamos valores
164     PORTB = bcd[numero_bcd];
165     estado_boton_anterior = estado_boton_actual;
166 }
167 }
```

Circuito físico

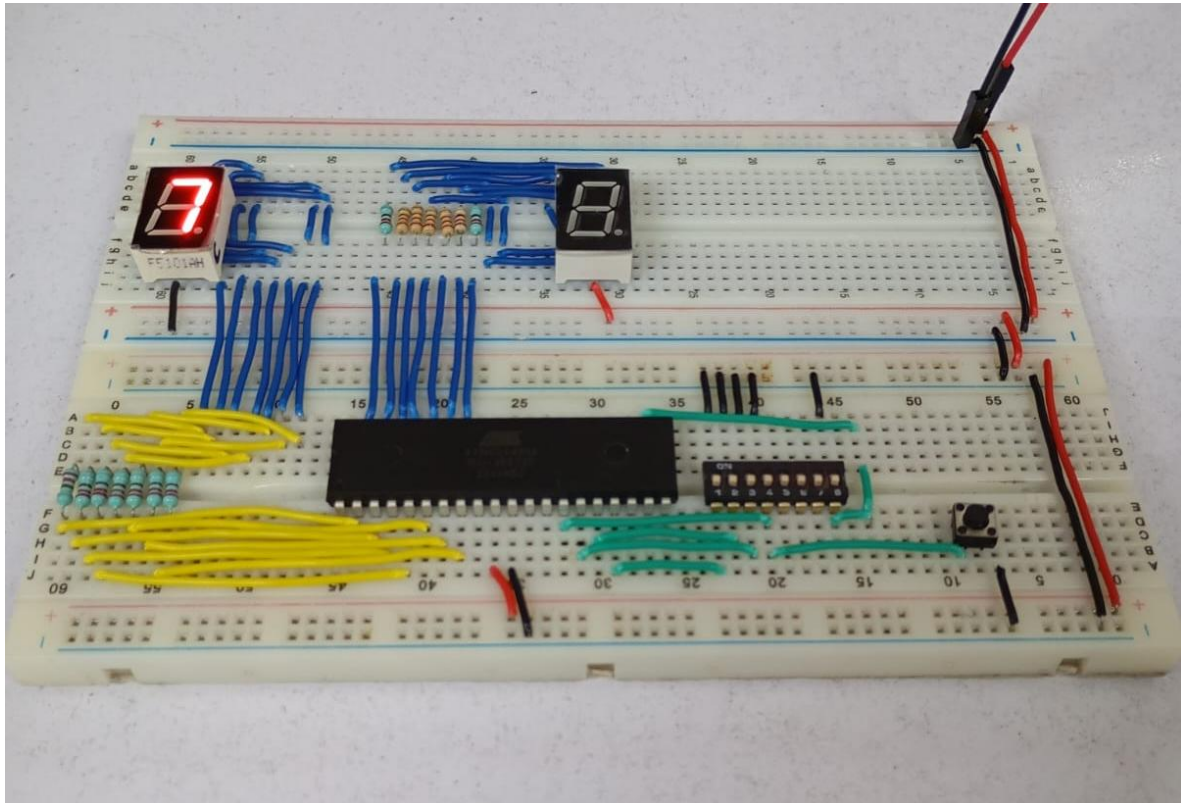


Imagen 3. Circuito físico de la práctica 6.

CONCLUSIÓN

- **García Escamilla Bryan Alexis**

Esta práctica se centra en despreciar los rebotes por completo, para ello ahora también se detecta en el flanco de bajada el cual en nuestro caso es de 0 a 1, es decir, ahora estamos detectando el cambio de valor al presionar el botón y al soltarlo, y es en este último en donde se coloca un retardo de 40 milisegundos (`delay_ms(40)`) suficiente para dejar pasar los rebotes del botón.

El código empleado en esta práctica es el que se usará cuando en el circuito se presente el uso del botón y así evitar comportamientos erróneos.

- **Meléndez Macedonio Rodrigo**

Para esta práctica se trabajó sobre el mismo circuito de las anteriores dos prácticas, pero ahora el enfoque fue en eliminar por completo el rebote que se hace al presionar el botón del circuito que armamos. Para ello, dentro del código con ayuda de un “if” ahora detectamos el flanco de bajada (0 a 1 en nuestro caso) y agregamos un pequeño retardo de 40 milisegundos con ayuda de la función “`delay_ms(40)`” anteriormente vista. Al añadir este retardo durante el flanco de bajada, pasa suficiente tiempo para que todos los rebotes ocurran al momento de

que presionamos el botón y de este modo, eliminamos por completo el mayor problema que teníamos con los contadores, el rebote.

BIBLIOGRAFÍA

- (s.f.). *Contact “Bounce”*. All about circuits. Recuperado el 10 de febrero de 2026.
<https://www.allaboutcircuits.com/textbook/digital/chpt-4/contact-bounce/>
- (s.f.). *A Guide to Debouncing*. Ganssle Group. Recuperado el 10 de febrero de 2026.
<http://www.ganssle.com/debouncing.htm>